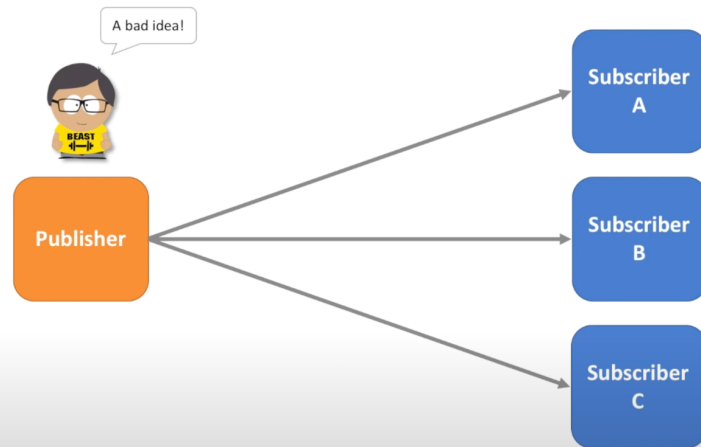


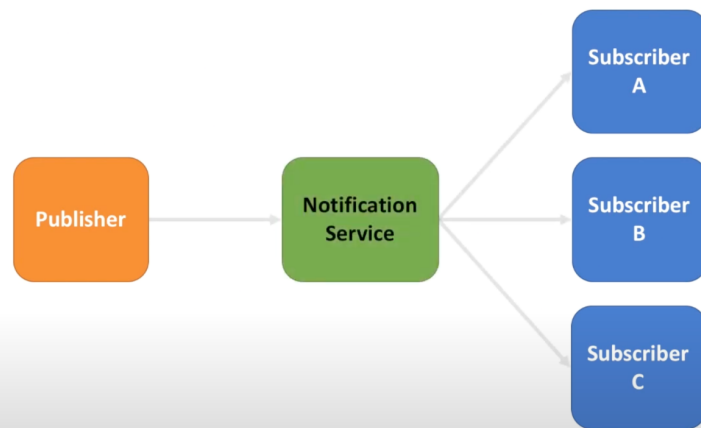
Notification Service

 System Design Interview - Notification Service

Problem Statement



Problem Statement



Requirements

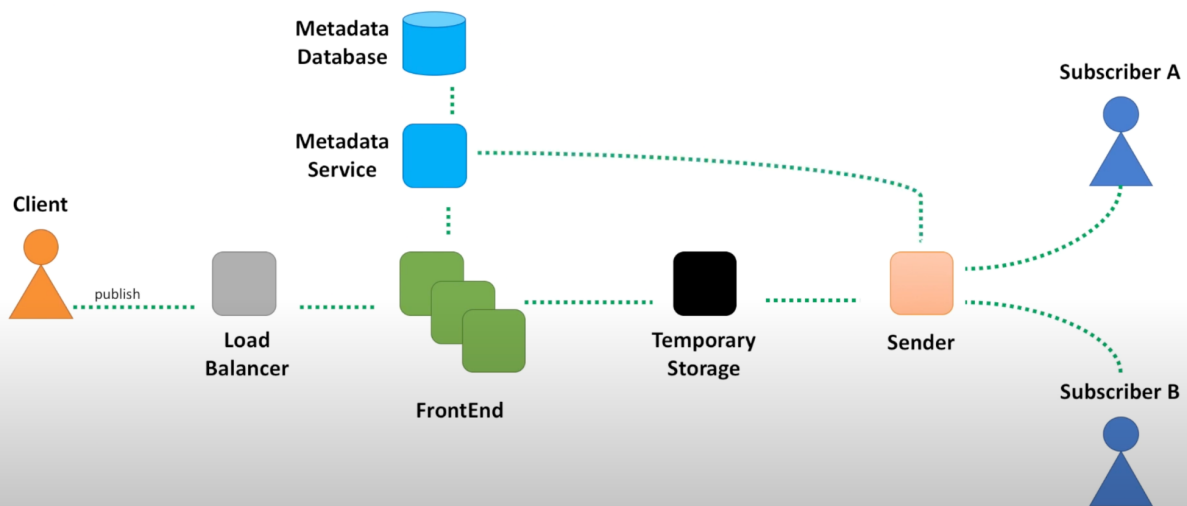
Functional

- `createTopic(topicName)`
- `publish(topicName, message)`
- `subscribe(topicName, endpoint)`

Non-Functional

- **Scalable** (supports an arbitrarily large number of topics, publishers and subscribers)
- **Highly Available** (survives hardware/network failures, no single point of failure)
- **Highly Performant** (keep end-to-end latency as low as possible)
- **Durable** (messages must not be lost, each subscriber must receive every message at least once)

High-level Architecture



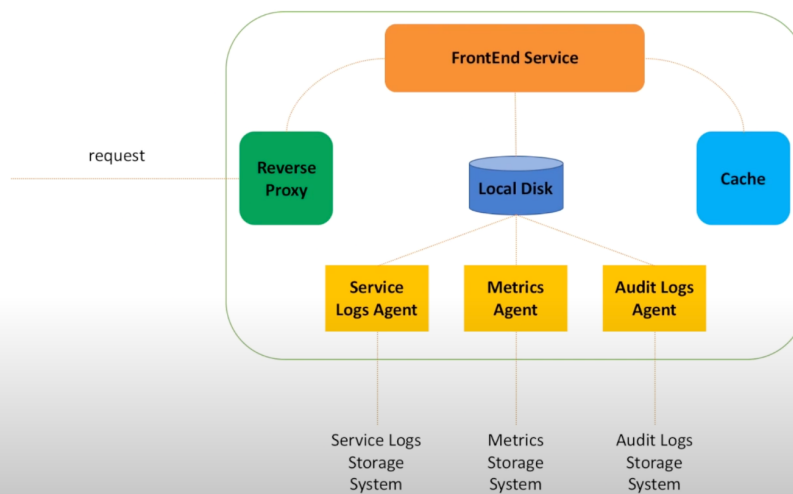
FrontEnd Service

- A lightweight web service
- Stateless service deployed across several data centers

Actions

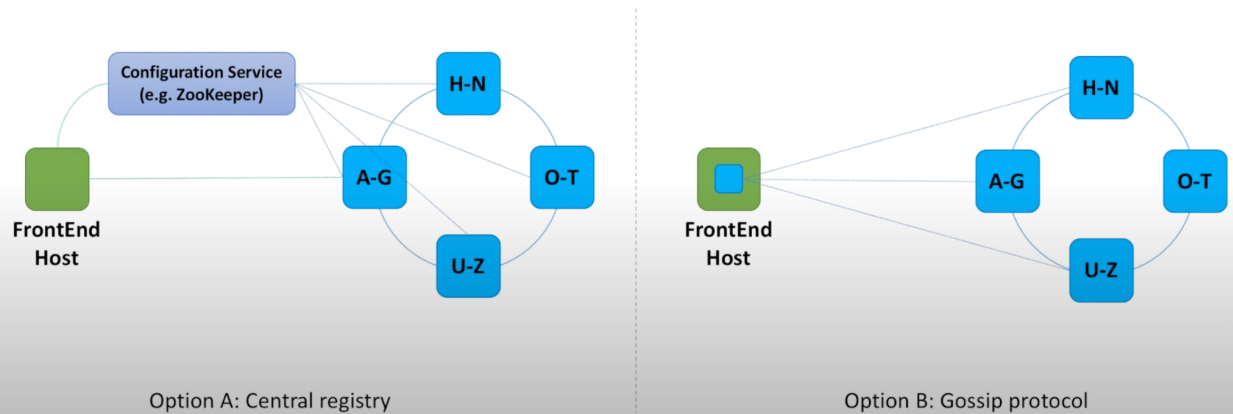
- Request validation
- Authentication/Authorization
- TLS (SSL) termination
- Server-side encryption
- Caching
- Rate limiting (throttling)
- Request dispatching
- Request deduplication
- Usage data collection

FrontEnd Service Host



Metadata Service

- A caching layer between the FrontEnd and a persistent storage
- Many reads, little writes



Temporary Storage

- Fast, highly-available and scalable web service
- Has to store messages for several days to handle unavailability of a subscriber

Several options can be considered, try to **lead the conversation** between **Interviewer** and **You**

Can we use a database?

Yes

SQL or NoSQL?

NoSQL

Any particular NoSQL type (column, document, key-value, graph) ?

Column or key-value

Any specific names?

Cassandra, Amazon DynamoDB

Databases

How about distributed memory caching system?

Sure, as long as persistence is supported

Any specific names?

Redis

In-memory storages

What other options do we have?

Message queue

Any specific names?

Apache Kafka, Amazon SQS

Message queues

What else?

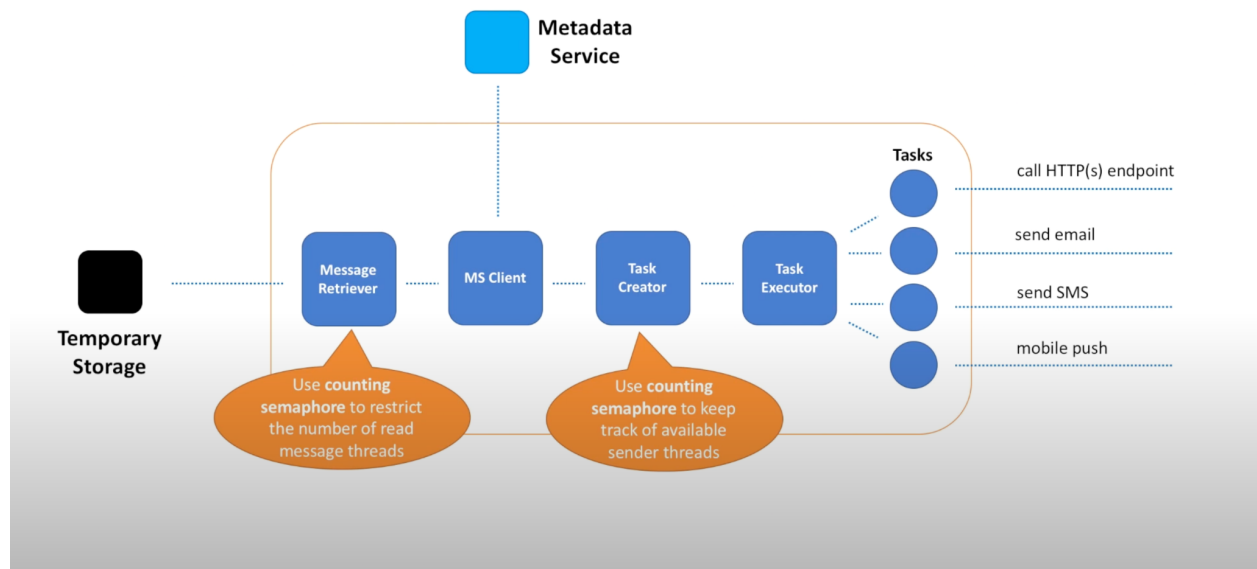
Stream-processing platforms

Any specific names?

Apache Kafka, Amazon Kinesis

Stream-processing platforms

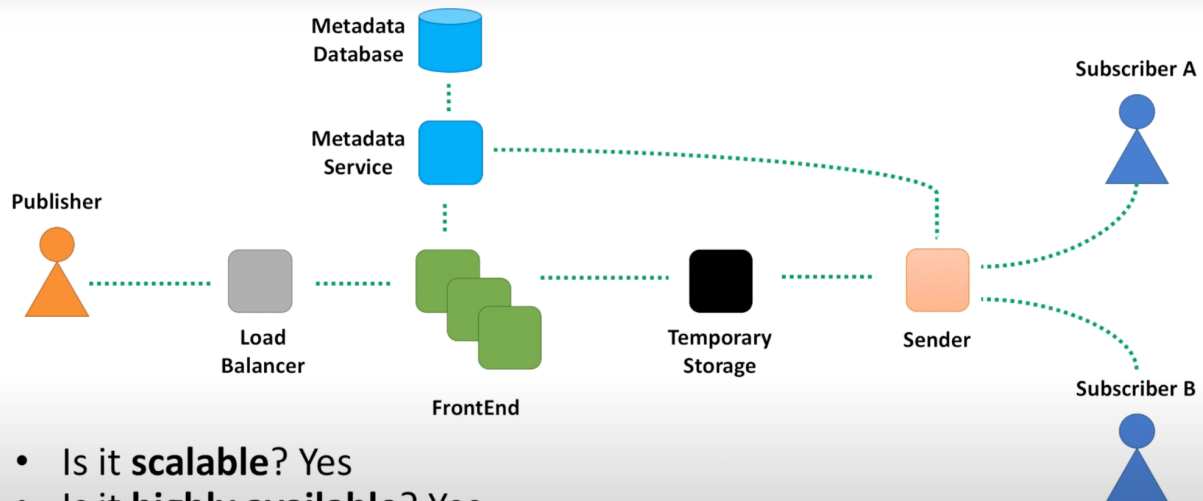
Sender



What else is important...

- Did we just build a solution for spammers?
- Duplicate messages
- Retry of the delivery attempt
- Message order
- Security
- Monitoring

One final look



- Is it **scalable**? Yes
- Is it **highly available**? Yes
- Is it **highly performant**? Yes
- Is it **durable**? Yes